



UNIVERSIDAD DE CARABOBO
FACULTAD EXPERIMENTAL DE CIENCIAS
Y TECNOLOGÍA
DEPARTAMENTO DE COMPUTACIÓN
ARQUITECTURA DEL COMPUTADOR



PRÁCTICA DE LABORATORIO #1

Estudio de la Recursividad e Iteración en Arquitectura MIPS32

Estudiantes:

Nathaly Campos *C.I. 32.460.314*
David Serrano *C.I. 31.000.672*

Fecha de Entrega:

25 de junio de 2026

1. ¿Cómo se implementa la recursividad en MIPS32? ¿Qué papel cumple la pila (\$sp)?

Forma de Implementación:

La recursividad en MIPS32 no se hace de manera automática como en los lenguajes de alto nivel (C o Java), sino que nos toca controlarla a nosotros mismos línea por línea usando subrutinas. Para lograrlo, se usa la instrucción `jal` (*Jump and Link*), la cual salta a la etiqueta de la función recursiva y al mismo tiempo guarda de forma automática la dirección de la siguiente línea del código en el registro `$ra` (*Return Address*). Como una función recursiva se llama a sí misma varias veces antes de terminar, si no guardamos el valor de `$ra` en algún lado, la siguiente llamada borraría la dirección de retorno original y el programa no sabría cómo regresar al `main`.

El papel de la Pila (\$sp):

Aquí es donde entra el registro Stack Pointer (`$sp`), que es un puntero que nos dice dónde está el tope de la pila en la memoria. Su función es ayudarnos a salvar y recuperar los datos para que no se borren en cada llamada:

- **Al iniciar la función:** Restamos espacio al registro `$sp` para apartar un pedazo de memoria en la RAM llamado marco de pila (*Stack Frame*). En ese espacio guardamos con la instrucción `sw` el registro `$ra` y cualquier argumento de entrada (como `$a0`) o registro que vayamos a modificar y que necesitemos recuperar intacto más adelante.
- **Al salir de la función:** Cuando el programa llega al caso base y empieza a devolverse, usamos la instrucción `lw` para sacar los valores viejos que guardamos en la memoria RAM y ponerlos otra vez en sus registros originales. Luego, le sumamos a `$sp` el mismo espacio que le quitamos al inicio para limpiar la pila antes de hacer el salto final con `jr $ra`.

2. ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

Riesgos comunes que pueden pasar:

- a) **Desbordamiento de Pila (Stack Overflow):** Como cada llamada recursiva obliga al programa a apartar un espacio de bytes en la pila de la memoria RAM, el consumo de memoria va creciendo a medida que el número que ingresa el usuario es más grande. Si el usuario mete un número excesivamente alto o si cometemos un error y la función se queda pegada llamándose a sí misma infinitamente, la pila se llenará por completo, chocará con otras secciones de la memoria y hará que el simulador se cierre con un error de ejecución.

- b) **Desbordamiento de Enteros (Integer Overflow):** En MIPS32 los registros guardan números enteros de 32 bits con signo. Si hacemos operaciones aritméticas que den un resultado que se salga de los límites permitidos, el bit de signo cambiará por error y el procesador interpretará un número positivo grande como si fuera un número negativo. Esto rompería las condiciones de parada del programa, haciendo que nunca se llegue al caso base.

Cómo solucionarlo o mitigarlo:

- **Validación de datos de entrada:** Podemos poner condiciones al principio del programa (usando instrucciones como `bltz` o `blez`) para verificar que el número ingresado no sea negativo o que no supere un límite máximo que consideremos seguro para la memoria.
- **Cambiar el enfoque del código:** Si sabemos que el programa va a manejar números muy grandes, lo mejor es no usar la recursividad y programar la solución usando un enfoque iterativo (con bucles), ya que los bucles no gastan espacio en la pila de memoria.

3. ¿Qué diferencias encontraste entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros?

Al comparar ambas formas de resolver el problema de la paridad, encontramos diferencias muy claras en cómo manejan la memoria y los registros del procesador. En cuanto al uso de memoria, el programa iterativo es sumamente eficiente porque no usa espacio extra en la pila (`$sp`), lo que significa que el consumo de memoria se mantiene constante sin importar qué tan grande sea el número ingresado. En cambio, el programa recursivo consume memoria de forma lineal, ya que por cada llamada que hace hacia atrás tiene que apartar un nuevo bloque en la memoria RAM para no perder los datos anteriores, lo que gasta muchos recursos si el número es alto.

Por el lado de los registros, la versión iterativa es mucho más rápida porque trabaja todo el tiempo de manera local con registros temporales (como `$t0` y `$t1`) directamente dentro del procesador y utiliza saltos condicionales simples para repetir el ciclo. Por el contrario, la versión recursiva se la pasa haciendo viajes constantes entre los registros de la CPU y la memoria RAM mediante instrucciones de carga y almacenamiento (`lw` y `sw`) para salvar la dirección de retorno (`$ra`) y el argumento actual (`$a0`), lo que causa que el procesador trabaje más lento debido a los tiempos de espera de la memoria externa.

4. ¿Qué diferencias encontraste entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32?

La diferencia principal entre los ejemplos que vemos en los libros de clase y un programa real y operativo en MIPS32 es que los libros suelen estar muy simplificados. El libro por lo general te muestra solamente la lógica matemática del algoritmo o el pedazo de código que hace la operación específica, dejando por fuera todo el entorno y la estructura necesaria para que la computadora de verdad pueda ejecutar el archivo de inicio a fin.

En un ejercicio real que queramos probar en un simulador como MARS, estamos obligados a escribir las directivas completas del ensamblador. Esto incluye armar la sección `.data` para definir los mensajes de texto que se van a mostrar en la pantalla, y la sección `.text` para indicarle al procesador dónde empiezan las instrucciones ejecutables. Además, los ejemplos del libro asumen que los datos ya están listos en los registros, mientras que en la práctica tenemos que programar la interfaz de usuario usando llamadas al sistema (`syscall`) para pedir el número por teclado, leerlo y acomodar las respuestas en la consola. Por último, un programa operativo necesita incluir validaciones contra números inválidos y una instrucción obligatoria de salida limpia (`li $v0, 10` con su respectivo `syscall`), ya que si no se pone, el procesador sigue leyendo memoria basura de largo y el simulador arroja un error físico.

5. Elaborar un tutorial de la ejecución paso a paso en MARS.

Dinámica de la Versión Iterativa (`paridad_iterativa.asm`):

- **Inicio:** Primero, el programa captura el número del usuario en el registro `$t0` (que usaremos como contador). Luego, usamos `li` para poner un 0 en `$t1` (nuestra variable de paridad) y un 1 en `$t2`. *Decisión de diseño:* Se decidió usar registros de tipo `$t` porque son temporales; sabemos que el procesador opera más rápido con ellos y, como no saltaremos a otras funciones, no corremos el riesgo de que se borren.
- **La barrera de entrada:** Al entrar al bucle, lo primero que evalúa la CPU es `blez $t0` (si el contador es menor o igual a cero, salta al final). *Decisión de diseño:* Se colocó la condición al inicio del ciclo y no al final para evitar que el programa haga operaciones innecesarias si el usuario ingresa un cero directamente.
- **El truco matemático:** La instrucción central es `sub $t1, $t2, $t1`, que

básicamente hace la resta $1 - \text{paridad}$. En el simulador se puede ver cómo `$t1` alterna mágicamente de 0 a 1 y de 1 a 0 en cada vuelta. *Decisión de diseño:* Esta resta directa evita tener que usar condicionales largos (tipo *if/else*) dentro del bucle, haciendo que el código corre a la máxima velocidad del hardware.

- **El avance:** Finalmente, con `addi $t0, $t0, -1` le restamos 1 a nuestro contador para acercarnos a cero, y un salto incondicional (`j`) nos devuelve al inicio del ciclo.

Dinámica de la Versión Recursiva (`paridad_recursiva.asm`):

- **El salto inicial:** En el bloque `main`, copiamos nuestro número a `$a0`. *Decisión de diseño:* La convención estándar de MIPS obliga a que los argumentos de las funciones viajen en los registros `$a`. Luego usamos `jal` para ir a la función; esta instrucción es vital porque hace el salto pero deja un "hilo de regreso" guardado en el registro `$ra`.
- **Cavando en la memoria (El restaura):** Apenas entra a la función, el procesador ejecuta `addi $sp, $sp, -8`. Aquí es donde vemos en MARS cómo el Stack Pointer (`$sp`) disminuye. ¿Por qué ocurre esto en este preciso momento? Porque la función está a punto de llamarse a sí misma de nuevo; si no guardamos ahora mismo nuestro `$ra` y nuestro `$a0` en la RAM usando `sw`, la nueva llamada los aplastará y el programa no sabrá cómo devolverse.
- **El caso base:** Se utiliza la instrucción `bgtz` para ver si el número ya llegó a 0. Cuando finalmente llega, el procesador carga un 0 en `$v0` (el registro oficial de respuestas) y salta directamente a la parte final (epílogo) para empezar a subir.
- **El ascenso:** En este punto empieza el efecto dominó hacia atrás. El procesador usa `lw` para leer la memoria y recuperar intactos los registros de la llamada anterior. Inmediatamente ejecuta `addi $sp, $sp, 8`. *Decisión de diseño:* Es obligatorio devolverle este espacio a la pila al subir; si no lo hacemos, estaríamos dejando "basura" en la memoria y causaríamos un desbordamiento.
- **Armando la respuesta:** Justo antes de usar `jr $ra` para volver a la etapa anterior, se ejecuta la resta `sub $v0, $t0, $v0`. El procesador toma el resultado que le entregó la recursión más profunda y lo invierte matemáticamente, armando la paridad final paso a paso mientras vuelve a la superficie.

6. ¿Cómo se Justifica la elección del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS?

- **Desde el punto de vista de la Eficiencia:** El enfoque **iterativo** es por mucho la mejor opción para esta práctica. MIPS32 es una arquitectura diseñada para ejecutar instrucciones simples muy rápido directamente en los registros del procesador. El código iterativo resuelve todo con un bucle usando registros locales, mientras que el código recursivo tiene que estar entrando y saliendo de la memoria RAM constantemente para guardar y sacar cosas de la pila, lo que genera retrasos y hace que gaste mucha más energía y tiempo de CPU.
- **Desde el punto de vista de la Claridad:** Es verdad que en papel la solución recursiva se ve más bonita y elegante porque copia exactamente la fórmula matemática ($1 - \text{paridad}(n - 1)$). Sin embargo, al momento de pasar esa idea a lenguaje ensamblador de bajo nivel, la recursividad se vuelve muy enredada porque estamos obligados a programar a mano el manejo de la pila, los prólogos y los epílogos. Por eso, el enfoque iterativo termina siendo mucho más fácil de leer, entender y corregir para nosotros.

7. Análisis y Discusión de los Resultados

Gracias a esta práctica pudimos ver la gran diferencia entre usar recursividad o bloques iterativos por la memoria y recursos que se consumen, por lo que llegamos a concluir y demostrar que:

- Hemos verificado que el código iterativo funcionara de manera perfecta y entregue el resultado de la paridad al instante sin importar si el número ingresado era pequeño o grande. Como este método mantiene el registro de la pila (\$sp) completamente intacto, demostró ser un programa seguro y estable.
- Por otro lado, al momento de ejecutar el código recursivo paso a paso pudimos ver exactamente cómo se van apilando los bloques de memoria uno encima de otro en la RAM. Esto nos hizo entender cómo funcionan las convenciones de llamadas y el resguardo de registros, pero también nos mostró lo frágil que es este método, ya que si probábamos con números muy grandes el simulador consumía demasiados recursos con riesgos a desbordamientos.
- En conclusión, esto nos enseña que cuando programamos en bajo nivel no podemos pensar solo en la matemática del problema, sino en las limitaciones físicas del hardware. Para operaciones sencillas como calcular una paridad, la iteración mediante bucles locales es siempre la opción correcta para no malgastar la memoria del computador.